Polynomial Inequalities in Creation of Abelian Square-Free Strings

Some preliminary examples

```
(* Inside the package, the main function structure is as follows. *)

vmmMakeA2FreeWord[str_String] := Module[{myP, myV, myRes},
    myP = vmmCreatePolynoms[str];
    myV = vmmPickVariables[str];
    myRes = vmmTestPolynom[myP, myV, 1];
    StringReplace[str, #] & /@ myRes];
```

```
myTestP = vmmPolyTests`Private`vmmCreatePolynoms["aXbYcZdTa"]
```

$$(a-T)(X-a)(b-X)(Y-b)(c-Y)(Z-c)(T-d)(d-Z)(-a+b-X+Y)(a-d+T-Z)(-b+c-X+Y)(-b+c-Y+Z)(-c+d+T-Z)(-c+d-Y+Z)$$
$$(-a-b+c-X+Y+Z)(a-c+d+T-Y-Z)(-b-c+d+T-Y+Z)(-b+c+d-X-Y+Z)(a-b-c+d+T-X-Y+Z)(-a-b+c+d+T-X-Y+Z)$$

```
vmmPolyTests`Private`vmmPickVariables["aXbYcZdTa"]
```

TXYZ

```
myRes = vmmPolyTests`Private`vmmTestPolynom[myTestP, "TXYZ", 1]
```

{{T → b, X → c, Y → a, Z → a}, {T → b, X → d, Y → a, Z → a}, {T → b, X → d, Y → a, Z → b},
{T → b, X → d, Y → d, Z → a}, {T → b, X → d, Y → d, Z → b}, {T → c, X → c, Y → a, Z → a}, {T → c, X → d, Y → a, Z → a}}

```
StringReplace["aXbYcZdTa", #] & /@ myRes
```

{acbacadba, adbacadba, adbacbdba, adbdcadba, adbdcbdba, acbacadca, adbacadca}

```
vmmMakeA2FreeWord["ABCD"]
```

{abac, abad, abca, abcb, abcd, abda, abdb, abdc, acab, acad, acba, acbc, acbd, acda, acdb, acdc, adab, adac, adba, adbc, adbd, adca, adcb,
adcd, babc, babd, baca, bacb, bacd, bada, badb, badc, bcab, bcac, bcad, bcba, bcbd, bcda, bcdb, bcdc, bdab, bdac, bdad, bdba, bdbc, bdca, bdcb,
bdcd, caba, cabc, cabd, cacb, cacd, cada, cadb, cadc, cbab, cbac, cbad, cbca, cbcd, cbda, cbdb, cbdc, cdab, cdac, cdad, cdba, cdbc, cdbd, cdca, cdcb,
daba, dabc, dabd, daca, dacb, dacd, dadb, dadc, dbab, dbac, dbad, dbca, dbcb, dbcd, dbda, dbdc, dcab, dcac, dcad, dcba, dcbc, dcbd, dcda, dcdb}

```
% // Length
```

96

```
vmmMakeA2FreeWord5let["ABC"]
```

{aba, abc, abd, abe, aca, acb, acd, ace, ada, adb, adc, ade, aea, aeb, aec, aed, bab, bac, bad, bae, bca, bcb, bcd, bce, bda, bdb, bdc, bde, bea, beb, bec, bed, cab, cac, cad, cae, cba, cbc, cbd, cbe,
cda, cdb, cdc, cde, cea, ceb, cec, ced, dab, dac, dad, dae, dba, dbc, dbd, dbe, dca, dcb, dcd, dce, dea, deb, dec, ded, eab, eac, ead, eae, eba, ebc, ebd, ebe, eca, ecb, ecd, ece, eda, edb, edc, ede}

```
% // Length
```

80

```
vmmMakeA2FreeWord3let["ABC"]
```

{aaa, aab, aac, aba, abb, abc, aca, acb, acc, baa, bab, bac, bba, bbb, bbc, bca, bcb, bcc, caa, cab, cac, cba, cbb, cbc, cca, ccb, ccc}

```
% // Length
```

27

```
vmmMakeA2FreeWord["abABCD", {{"A", "B", "C", "D", "a", "b", "c"}}]
```

{abacab, abacba, abacbc, abcaba, abcbab}

```
vmmMakeA2FreeWord["abABCD", {{"A", "B", "C", "D", "a", "b", "c"}}]
```

{abacab, abacba, abacbc, abcaba, abcbab}

```
vmmMakeA2FreeWord["abABCDE", {{"A", "B", "C", "D", "E", "a", "b", "c"}}]
```

{abacaba, abacbab, abcbabc}

There are no a-2-free words over 3 letters of length 8. Note that here we do not allow factors *aa*, *bb*, *cc*:

```
vmmMakeA2FreeWord["abABCDEF", {{"A", "B", "C", "D", "E", "F", "a", "b", "c"}}]
```

{}

If we allow factors *aa*, *bb*, *cc* (and *aaa*, *bbb*, *ccc*) – but no longer abelian squares – the situation is very different:

```
vmmMakeA2FreeWord3let["abABCDEF", {{"A", "B", "C", "D", "E", "F", "a", "b", "c"}}]
```

{abaaacaa, abaaacab, abaaacba, abaaacbb, abaaacbc, abaaaccb, abaaaccc, abaacaaa, abaacaab, abaacaba, abaacabb, abaacabc, abaacbaa, abaacbab, abaacbac, abaacbba, abaacbbb, abaacbca,
abaacbcc, abaaccba, abaaccbb, abaaccbc, abaaccca, abaacccb, abacaaab, abacaaba, abacaabb, abacabaa, abacabbb, abacabbc, abacbaaa, abacbabb, abacbbaa, abacbbab, abacbbba,
abacbbbc, abacbcca, abacbccc, abaccbba, abaccbbb, abaccbca, abaccbcc, abacccaa, abacccab, abacccba, abacccbb, abaccccbc, abbbaaab, abbbaaac, abbbaaca, abbbaacb, abbbaacc,
abbbacaa, abbbacab, abbbacba, abbbacbb, abbbacbc, abbbaccb, abbbaccc, abbbcaaa, abbbcaab, abbbcaba, abbbcabb, abbbcabc, abbbcacb, abbbcacc, abbbcbaa, abbbcbab,
abbbcbac, abbbcbba, abbbcbbb, abbbccaa, abbbccab, abbbccac, abbbccca, abbbcccb, abbcaaab, abbcaaac, abbcaaba, abbcaabb, abbcabaa, abbcabbb, abbcabcc, abbcacbc,
abbcaccb, abbcaccc, abbcbaaa, abbcbaac, abbcbabb, abbcbaca, abbcbacc, abbcbbaa, abbcbbab, abbcbbba, abbccaaa, abbccaab, abbccaba, abbccabc, abbccacb, abbccacc,
abbcccaa, abbcccab, abbcccac, abbcccba, abbccctb, abcaaaba, abcaaabb, abcaaacc, abcaabaa, abcaabbb, abcaabbc, abcabaaa, abcabbba, abcabbbc, abcabbcb, abcabbcc, abcaccba,
abcaccbb, abcaccbc, abcacccb, abcbaaab, abcbaaac, abcbaaca, abcbaacc, abcbabbb, abcbabcc, abcbbaaa, abcbbaac, abcbbabb, abcbbaca, abcbbacc, abcbbbaa, abcbbbab,
abccaaab, abccaaac, abccaaba, abccaabb, abccaabc, abccacba, abccacbb, abccaccc, abcccaaa, abcccaab, abcccaba, abcccabb, abcccacc, abcccbaa, abcccbab, abcccbba, abcccbbb}